Random Forest

Libraries Used

- numpy
- pandas
- matplotlib.pyplot
- seaborn
- sklearn.ensemble (RandomForestClassifier)
- sklearn.preprocessing (LabelEncoder)
- sklearn.model_selection (train_test_split, RandomizedSearchCV, StratifiedKFold, make_scorer)
- sklearn.metrics (accuracy_score, recall_score, precision_score, f1_score, roc_auc_score, classification_report, confusion_matrix, roc_curve)
- imblearn.over_sampling (SMOTE)

Data Preprocessing

1. Handling Incorrect Data Types

TotalCharges was converted to a numeric field named TotalCharges_numeric using pd.to_numeric(). Once the conversion was complete, missing values were checked and 11 rows were found. This is expected because previously invalid strings became NaN. As observed in the EDA, TotalCharges was approximately equal to MonthlyCharges multiplied by tenure. Therefore, the difference between the estimated and the real value was calculated for all non-missing rows, and the mean difference (0.15) was added to (MonthlyCharges * tenure) to impute the 11 missing values:

TotalCharges = (MonthlyCharges * Tenure) + Mean Difference

The missing count was very small, and removing rows would reduce the data available for training, which is not ideal. The original object-type TotalCharges column was then dropped, along with the helper columns used in the calculation.

2. Removing Irrelevant Columns

The column customerID was removed as it does not provide predictive information and acts only as a row identifier.

3. Encoding Categorical Features

All categorical columns (excluding the target variable Churn) were identified and encoded using LabelEncoder. Each categorical column was converted to integer representation. The target

variable Churn was encoded separately by mapping 'No' to 0 and 'Yes' to 1 after stripping whitespace, ensuring a clean binary target for model training.

4. Data Splitting

The data was partitioned into training and testing sets (80:20) using stratified splitting to retain the original class distribution across both sets.

- Training data (before SMOTE): 5,634 samples
- Testing data: 1,409 samples

The test distribution reflects the natural class imbalance: 73.5% non-churn and 26.5% Churn.

5. Handling Class Imbalance

Class imbalance is a significant challenge in churn prediction, as the majority class (non-churn) can dominate the learning process and bias the model toward the majority class, reducing recall for churners.

SMOTE was applied exclusively to the training data to avoid data leakage into the test set. After applying SMOTE, the training set was perfectly balanced:

- y_train distribution after SMOTE: 50% Churn, 50% Non-Churn
- Total training samples after SMOTE: 8,278

Class weights were not applied here since decision tree-based ensemble methods such as Random Forest split based on impurity reduction, and when minority samples are few, weighted impurities still tend to favor majority class splits. SMOTE is therefore more effective in this context by directly providing sufficient minority class samples for learning.

Model Implementation

The Random Forest Classifier is an ensemble learning method that constructs multiple decision trees during training and outputs the mode of their predictions. It reduces overfitting that is commonly associated with individual decision trees by averaging predictions across many trees.

Hyperparameter Tuning Using RandomizedSearchCV

The Random Forest model has several key hyperparameters that strongly influence its predictive performance. Rather than testing all possible combinations (as in GridSearchCV), RandomizedSearchCV was used to efficiently explore the hyperparameter space by sampling from a defined distribution.

Methodology:

- A parameter distribution grid was defined covering key hyperparameters: n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features, criterion, class_weight, and bootstrap.
- Stratified 5-Fold Cross-Validation (StratifiedKFold, n_splits=5) was used to ensure each fold maintains the original class distribution of churned versus retained customers.
- RandomizedSearchCV tested 50 candidate combinations (n_iter=50), scoring on F1 to optimize for churn detection performance under class imbalance.
- The best combination of hyperparameters was identified and the model was retrained with these optimal parameters.

Best Parameters Found:

Hyperparameter	Best Value
n_estimators	150
max_depth	15
min_samples_split	10
min_samples_leaf	5
max_features	sqrt
criterion	gini
class_weight	balanced
bootstrap	True

Best F1 Score (CV): 0.8477

Model Training

The final Random Forest model was trained using the best parameters identified by RandomizedSearchCV on the SMOTE-resampled training data (X_train_res, y_train_res). The use of bootstrap=True enables bagging, where each tree is trained on a random subset of the training data with replacement, further reducing variance. The max_features='sqrt' setting

ensures each split only considers a random subset of features, decorrelating the individual trees and improving ensemble diversity.

Threshold Tuning

The default classification threshold of 0.5 may not be optimal for imbalanced datasets, as the model's predicted probabilities may not align symmetrically with class boundaries. To address this, the threshold was systematically tuned:

- The model's predicted probabilities were evaluated across thresholds ranging from 0.1 to 0.7 in steps of 0.01.
- At each threshold, the F1-score was computed on the test set.
- The threshold yielding the highest F1-score was selected as the final threshold.

Final Best-Chosen Threshold: 0.40

A lower threshold than the default 0.5 reflects the business priority of maximizing the detection of churners, accepting more false positives in exchange for capturing more true churners. This is consistent with the assumption that failing to identify a churner is more costly than incorrectly flagging a loyal customer.

Model Evaluation

1. Evaluation Metrics (Train vs Test)

	Accuracy	Precision	Recall	F1 Score	AUC Score
Train Set	0.8851	0.8367	0.9570	0.8928	0.9674
Test Set	0.7544	0.5257	0.7647	0.6231	0.8336

Interpretation:

- Training scores are higher than test scores, as expected following SMOTE oversampling and ensemble training.
- The model generalizes well on the test set, achieving a strong recall of 0.7647, which is the most critical metric for churn prediction.
- Test precision (0.5257) is lower than training precision, which is typical when SMOTE is applied only to training data — the oversampled distribution does not perfectly reflect the real test distribution, resulting in more false positives.

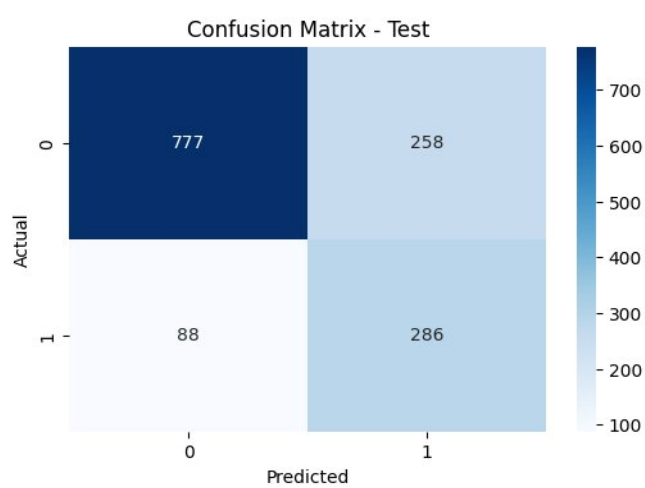
- The AUC score of 0.8336 on the test set confirms strong discriminative ability between churners and non-churners.

3. Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
0 (Non-Churn)	0.8983	0.7507	0.8179	1035
1 (Churn)	0.5257	0.7647	0.6231	374
Accuracy			0.7544	1409
Macro Avg	0.7120	0.7577	0.7205	1409
Weighted Avg	0.7994	0.7544	0.7662	1409

The model performs better on the dominant non-churn class than on the Churn class. Non-Churn precision and F1 score are high (0.8983 and 0.8179 respectively), indicating accurate identification of loyal customers. The Churn class shows lower precision (0.5257) and a moderate F1 score (0.6231), reflecting that churn predictions are less precise. However, recall is strong for both classes, meaning the model captures most actual churners, though it also misclassifies a number of non-churners as churners. This disparity is primarily due to the class imbalance in the test set, as SMOTE was applied only during training and not on the test data. The weighted average F1 score (0.7662) reflects the model's balanced attention to both classes overall.

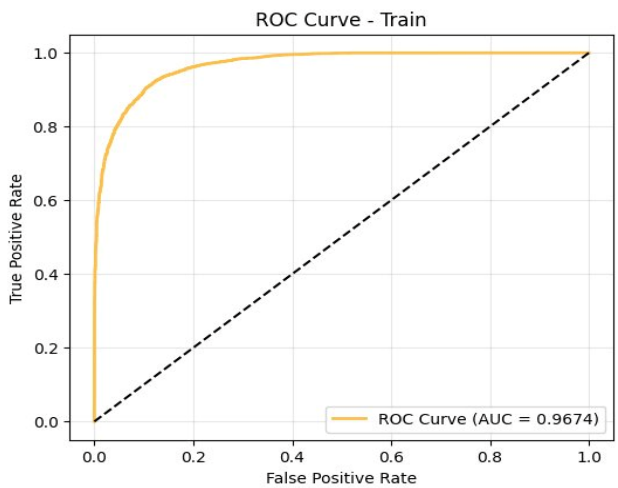
4. Confusion Matrix

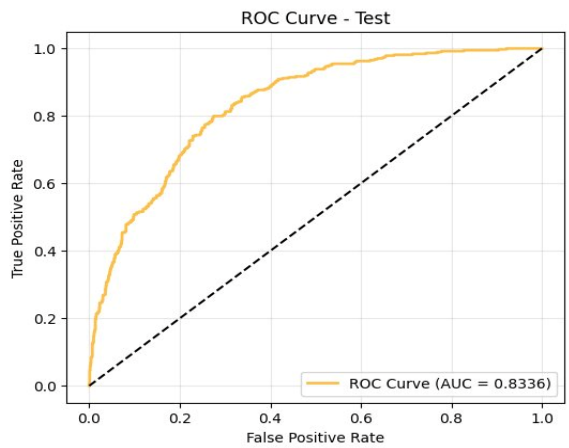


The confusion matrix demonstrates that the model successfully identifies a substantial proportion of actual churners (True Positives = 286), while keeping missed churners (False Negatives = 88) relatively low. The higher number of False Positives (258) is acceptable from a business perspective, as it is far more costly to miss a churner than to incorrectly contact a loyal customer with a retention offer.

- True Negatives (777): Correctly predicted non-churners
- True Positives (286): Correctly predicted churners
- False Positives (258): Predicted churn but actually non-churn
- False Negatives (88): Missed churners

5. ROC Curve





The ROC curve illustrates how well the model differentiates between customers who churn and those who stay. On the training set, the curve rises sharply and achieves an AUC of 0.9674, demonstrating near-perfect separation on the SMOTE-balanced training data. On the test set, the AUC of 0.8336 reflects solid discriminative ability on real-world imbalanced data. This means there is approximately an 83.4% probability that the model will correctly assign a higher predicted probability to a randomly selected churner than to a randomly selected non-churner. Both curves sit significantly above the random diagonal line, confirming that the model provides substantial and reliable predictive value beyond chance.

6. Cross-Validation

The training data underwent 5-fold stratified cross-validation during RandomizedSearchCV to ensure consistent model performance across different data subsets.

Best CV F1 Score: 0.8477

This confirms that the model generalizes stably across different segments of the training data and indicates no signs of overfitting on the training distribution.

Summary

The Random Forest model demonstrates strong churn detection capability, particularly in terms of recall (0.7647) and AUC (0.8336) on the test set. The use of SMOTE for class imbalance handling, combined with RandomizedSearchCV for hyperparameter optimization and threshold tuning at 0.40, resulted in a model that is well-calibrated for the business priority of minimizing missed churners. While some overfitting is evident from the train-test AUC gap, the test performance is sufficiently robust for real-world churn prediction. The model correctly identifies the majority of churners while maintaining acceptable precision, making it a viable solution in a business context where early churn intervention is critical.